

有灵Lite&Pro 数字人驱动 SDK 使用说明

简介

本 JS-SDK（以下简称 SDK）是魔法星云开放平台3D虚拟人角色驱动能力服务向网页开发者提供的网页开发工具。SDK提供了虚拟人渲染、语音合成、动作控制等核心功能。

版本记录

🌐 [星云 lite Web SDK 版本 changelog](#)

版本	更新时间	说明	更新人
0.0.1	2025.07.03	试运行版本，包含数字人渲染和 widget 功能	
0.1.0-alpha.15	2025.08.26		
0.1.0-alpha.18	2025.08.25	1. 删除 <code>init</code> 方法的 <code>onClose</code> 参数 2. 新增 <code>onStatusChange</code>	 方超
0.1.0-alpha.63	2025.10.24	webgl 上下文丢失时，增加二次初始化失败处理	 李冬冬
0.1.0-alpha.75	2025.11.26	pcm 播放器外网资源集成到 sdk 内部	 郑林雄
0.1.0-alpha.95	2025.12.19	更改 webgl 判断逻辑，修复机器人报错未出现	 郑林雄

使用说明

准备工作

浏览器环境使用

1. 在页面中引入以下依赖：

依赖包	版本	CDN地址
xmov-avat95	0.1.0-alpha.95	https://media.youyan.xyz/youling-lite-sdk/index.umd.0.1.0-alpha.95.js

2. HTML结构示例:

代码块

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <body>
4   <div style="width: 400px;height: 600px">
5     <div id="sdk"></div>
6   </div>
7   <script src="https://media.youyan.xyz/youling-lite-sdk/index.umd.0.1.0-alpha.95.js"></script>
8 </body>
9 </html>
```

NPM安装

代码块

```
1 $ npm config set registry http://npm.xmov.ai
2 $ npm install @xmov/avatar
```

初始化

代码块

```
1 import XmovAvatar from '@xmov/avatar'
2
3 const LiteSDK = new XmovAvatar({
4   containerId: '#sdk',
5   appId: '',
6   appSecret: '',
7   gatewayServer: '',
8   headers: {
9     'Authorization': '888jn',
10  },
11  config: config,
12  hardwareAcceleration: "prefer-hardware", // 开启硬件加速
13  // 自定义渲染器, 传递该方法, 所有事件sdk均返回, 由该方法定义所以类型事件的实现逻辑
14  onWidgetEvent(data) {
15    // 处理widget事件
16    console.log('Widget事件:', data)
17  },
18  // 代理渲染器, sdk默认支持subtitle_on、subtitle_off和widget_pic事件。通过代理,
```

```

19 // 可以修改默认事件，业务侧也可实现各种其他事件。
20 proxyWidget: {
21   "widget_slideshow": (data: any) => {
22     console.log("widget_slideshow", data);
23   },
24   "widget_video": (data: any) => {
25     console.log("widget_video", data);
26   },
27 },
28 onNetworkInfo(networkInfo) {
29   console.log('networkInfo:', networkInfo)
30 },
31 onMessage(message) {
32   console.log('SDK message:', message);
33 },
34 onStateChange(state: string) {
35   console.log('SDK State Change:', state);
36 },
37 onStatusChange(status) {
38   console.log('SDK Status Change:', status);
39 },
40 onStateRenderChange(state: string, duration: number) {
41   console.log('SDK State Change Render:', state, duration);
42 },
43 onStartSessionWarning(message: Object) {
44   console.log("onStartSessionWarning=====", message);
45 },
46 onVoiceStateChange(status:string) {
47   console.log("sdk voice status", status);
48 },
49 enableLogger: false, // 不展示sdk log, 默认为false
50 enableClientInterrupt: false,
51 })

```

初始化参数

参数名	类型	参数	必填	说明
containerId	string		是	容器元素ID
appId	string		是	数字人appId（从业务系统获取）
appSecret	string		是	数字人secretId（从业务系统获取）

gatewayServer	string		是	数字人服务接口完整路径
hardwareAcceleration	string		否	是否开启硬件加速 默认: 'default' 硬件加速: "prefer-hardware" CPU软解: "prefer-software"
headers	Object		否	自定义请求头
onWidgetEvent	function		否	Widget事件回调函数
proxyWidget	Object		否	Sdk内默认支持subtitle_on、 subtitle_off和widget_pic事件。 用户可以通过两种方式重写默认事件及支持自定义事件。 1. 使用onWidgetEvent代理所有事件，sdk会将事件全部抛出。 2. 使用proxyWidget代理事件，用户可根据需求实现自定义事件或代理默认事件。 事件系统优先级如下： onWidgetEvent > proxyWidget > 默认事件。
onVoiceStateChange	Function	Status: 音频播放状态 "start": 开始播放 "end": 播放结束	否	监控sdk音频播放状态
onNetworkInfo	function	networkInfo: SDKNetworkInfo (见: 参数说明)	否	当前网络状况
onMessage	function	message: SDKMessage (见: 参数说明)	是	SDK 消息
onStateChange	function	state: string (国【动作线】有 灵Lite&Pro状态说明)	否	监听虚拟人状态变化
onStatusChange	function	status: SDKStatus (见: 参数说明)		
onStateRenderChange	function		否	监听虚拟人状态变化耗时 从发送action到状态首帧渲染

onStartSession Warning	function	Message: Object	否	抛出数字人配置不正确的警告信息
enableLogger	boolean		否	是否打印sdk日志
enableClientInterrupt	boolean		否	是否需要端上打断speak，而不是服务端返回数据打断
config	Object	代码块<pre>1 { 2 "layout": { 3 4 "container": { 5 6 "size": [7 1440, 8 810 9], 10 }, 11 "avatar": { 12 "v_align": 13 "center", 14 "h_align": 15 "middle", 16 "scale": 0.4, 17 "offset_x": 0, 18 "offset_y": 0 19 } 20 }, 21 "walk_config": 22 { 23 "min_x_offset": 24 -500, 25 "max_x_offset": 26 500,</pre>		layout: walk_config:

		20 "walk_points": { 21 "A": -500, 22 "B": -400, 23 "C": -300, 24 "D": -200, 25 "E": -100, 26 "F": 0, 27 "G": 100, 28 "H": 200, 29 "I": 300, 30 "J": 400, 31 "K": 500 32 }, 33 "init_point": 0 34 } 35 }		
--	--	---	--	--

参数说明

代码块

```
1  enum EErrorCode {
2      // 容器不存在
3      CONTAINER_NOT_FOUND = 10001,
4      // socket连接错误
5      CONNECT_SOCKET_ERROR = 10002,
6      // 会话错误, start_session进入catch (/api/session的接口数据异常, 均使用
response.error_code)
7      START_SESSION_ERROR = 10003,
8      // 会话错误, stop_session进入catch
9      STOP_SESSION_ERROR = 10004,
10
```

```

11 VIDEO_FRAME_EXTRACT_ERROR = 20001, // 视频抽帧错误
12 INIT_WORKER_ERROR = 20002, // 初始化视频抽帧WORKER错误
13 PROCESS_VIDEO_STREAM_ERROR = 20003, // 抽帧视频流处理错误
14 FACE_PROCESSING_ERROR = 20004, // 表情处理错误
15
16 BACKGROUND_IMAGE_LOAD_ERROR = 30001, // 背景图片加载错误
17 FACE_BIN_LOAD_ERROR = 30002, // 表情数据加载错误
18 INVALID_BODY_NAME = 30003, // body数据无Name
19 VIDEO_DOWNLOAD_ERROR = 30004, // 视频下载错误
20
21 AUDIO_DECODE_ERROR = 40001, // 音频解码错误
22 FACE_DECODE_ERROR = 40002, // 表情解码错误
23 VIDEO_DECODE_ERROR = 40003, // 身体视频解码错误
24 EVENT_DECODE_ERROR = 40004, // 事件解码错误
25 INVALID_DATA_STRUCTURE = 40005, // ttsa返回数据类型错误, 非audio、body、face、
event等
26 TTSA_ERROR = 40006, // ttsa下行发送异常信息
27
28 NETWORK_DOWN = 50001, // 离线模式
29 NETWORK_UP = 50002, // 在线模式
30 NETWORK_RETRY = 50003, // 网络重试
31 NETWORK_BREAK = 50004, // 网络断开
32 }
33
34 interface SDKMessage {
35     code: EErrorCode
36     message: string
37     timestamp: number
38     originalError?: string
39 }
40
41 interface SDKNetworkInfo {
42     rtt: number // 延迟, 毫秒
43     downlink: number // 下载速率 (MB/s)
44 }
45
46 enum SDKStatus {
47     online = 0,
48     offline = 1,
49     network_on = 2,
50     network_off = 3,
51     close = 4,
52     invisible = 5,
53     visible = 6,
54     stopped = 7,
55 }

```

Widget事件类型

Widget事件回调函数接收的数据格式如下：

代码块

```
1  interface IData<T, K> {
2      type: T
3      data: K
4  }
5
6  // Widget类型定义
7  type TWidgetType =
8      | 'widget_pic'          // 单个图片
9      | 'widget_slideshow'    // 轮播
10     | 'widget_subtitle'     // 字幕
11     | 'widget_text'         // 文本
12     | 'widget_video'        // 视频
13
14  // 各类型对应的数据结构
15  interface IWidgetPic {
16      axis_id: number
17      image: string
18  }
19
20  interface IWidgetSlideshow {
21      axis_id: number
22      images: Array<{
23          image: string
24      }>
25  }
26
27  interface IWidgetSubtitle {
28      subtitle: string
29  }
30
31  interface IWidgetText {
32      axis_id: number
33      text_content: string
34  }
35
36  interface IWidgetVideo {
37      axis_id: number
38      video: string
39      cover: string
40  }
```


API方法

init()

初始化SDK，加载必要资源。

代码块

```
1  async init({
2    initModel: 'normal' | 'invisible'
3    onDownloadProgress?: (progress: number) => void,
4    onError: (error: SDKError) => void,
5    // socket断开
6    onClose: () => void,
7  }): Promise<void>
```

参数说明：

参数名	类型	参数	必填	说明
onDownloadProgress	function		是	资源下载进度回调，progress取值范围[0, 100]，首次连接bin资源或首个视频资源加载失败时，进度均不会达到100，且内部会调用stopSession。防止用户无法重新连接。
initModel	string	'normal' 'invisible' normal: 正常初始化 invisible: 隐身初始化	否	初始化模式，两种模式：正常和隐身

idle()

待机

代码块

```
1  idle(): void
```

speak()

控制虚拟人说话。

代码块

```
1 speak(ssml: string, is_start: boolean, is_end: boolean): void
```

参数说明：

- `ssml` : SSML格式的语音内容

faceDetect()

人脸检测

代码块

```
1 faceDetect(): void
```

wakeUp()

唤醒

代码块

```
1 wakeUp(): void
```

listen()

倾听

代码块

```
1 listen(): void
```

think()

思考

代码块

```
1 think(): void
```

interactiveidle()

打断当前状态，回到待机状态

代码块

```
1 interactiveidle(): void
```

skill()

表演

代码块

```
1 skill(action_semantic: string): void
```

touchReact()

点击互动

代码块

```
1 touchReact(): void
```

exitInteraction()

退出响应

代码块

```
1 exitInteraction(): void
```

stop()

停止当前内容播放。

代码块

```
1 stop(): void
```

setVolume()

控制声音 取值范围0->1 0:静音 1:最大音量

代码块
`setVolume(volume: number): void`

destroy()

销毁SDK实例，断开连接。

代码块

```
1  destroy(): void
```

showDebugInfo() / hideDebugInfo()

显示/隐藏调试信息。

代码块

```
1  showDebugInfo(): void
2  hideDebugInfo(): void
```

offlineMode() / onlineMode()

主动切换离线/在线。

代码块

```
1  offlineMode(): void
2  onlineMode(): void
```

switchInvisibleMode

主动切换隐身/在线

代码块

```
1  switchInvisibleMode()
```

changeAvatarVisible

主动UI层面隐藏和显示数字人，cnanvas display: block/none

代码块

```
1  changeAvatarVisible(visible: boolean):void
```

interrupt

主动打断数字人speak状态

代码块

```
1 interrupt(type: string):void
```

错误处理

SDK使用 `SDKError` 接口定义错误信息：

代码块

```
1 interface SDKError {  
2     code: number // 错误码  
3     message: string // 错误信息  
4     timestamp: number // 错误发生时间戳  
5     originalError: any // 原始错误信息  
6 }
```

注意事项

1. 确保容器元素具有明确的宽高，否则可能影响渲染效果
2. 初始化时必须提供所有必填参数
3. 使用 `destroy()` 方法时会清理所有资源并断开连接
4. 建议在开发环境使用 `showDebugInfo()` 辅助调试

最佳实践

1. 在页面卸载前调用 `destroy()` 方法清理资源
2. 合理使用错误回调处理异常情况
3. 使用SSML格式控制虚拟人说话效果
4. 需要自定义Widget行为时实现 `onWidgetEvent` 回调或者proxyWidget补齐配置

数字人自定义事件功能允许您在数字人说话过程中，通过SSML中的 `<uievent>` 标签触发自定义UI事件，实现**多模态内容展示**。例如：展示图片、播放视频、显示链接、展示3D模型等，能更好的提高数字人交互的内容丰富度。[📖 自定义事件详细教程](#)

5. 使用KA插件时，建议先测试动作意图识别效果，根据需要调整配置参数